CODAFRIQA

TECHNICAL DOCUMENTATION

AI Customer Support Chatbot

System Architecture, Functional Capabilities & Deployment Guide

---

Prepared by

Arnold Niyonkuru

Document Version 1.0

August 31, 2026

Table of Contents

1. Executive Summary

This project delivers a full-stack, enterprise-grade AI Customer Support Chatbot system designed for scalable, context-aware customer interactions and support operations. The platform integrates a modern single-page frontend application with a robust micro-framework backend, backed by persistent Relational Database Management System (RDBMS) structures and high-performance vector retrieval capabilities.

Frontend	Backend	Data & AI Layer
Vue 3 + Vite	Spring Boot 3.3+ (Java 17/21)	PostgreSQL + pgvector
Pinia State Management	Spring Data JPA / Hibernate	Google Gemini API (LLM + Embeddings)

1.1 Core System Stack

Layer	Framework & Technology	Key Capabilities
Frontend	Vue 3, Vite, Tailwind CSS	Single-page architecture, Composition API, script setup
State Management	Pinia (6 modules)	Centralized reactive stores (authStore, chatStore, maintenanceStore, ...)
Backend API	Spring Boot 3.3+ (Java 17/21)	RESTful API, service layer architecture, global exception handling
Security	Spring Security	Role-based access control (RBAC), endpoint authentication, CORS mappings
AI Integration	Spring AI, Google Gemini API	gemini-3.6-flash (LLM generation), text-embedding-004 (embeddings)
Persistence	PostgreSQL + pgvector	Relational entity management, cosine similarity vector storage

2. System Architecture & Component Deep Dive

The system follows a three-tier architecture, separating presentation, application logic, and data/AI concerns for maintainability and scale.

Layer	Technology	Responsibility
Presentation	Vue 3 Frontend	User/Agent Workspace, Pinia State Management, Axios API Layer
Application	Spring Boot Backend	Security/CORS, Service Logic Layer, Spring AI / Gemini integration, JPA/Hibernate ORM
Data & AI	PostgreSQL + pgvector	users, chat_sessions, chat_messages, support_tickets, vector_store, system_tools

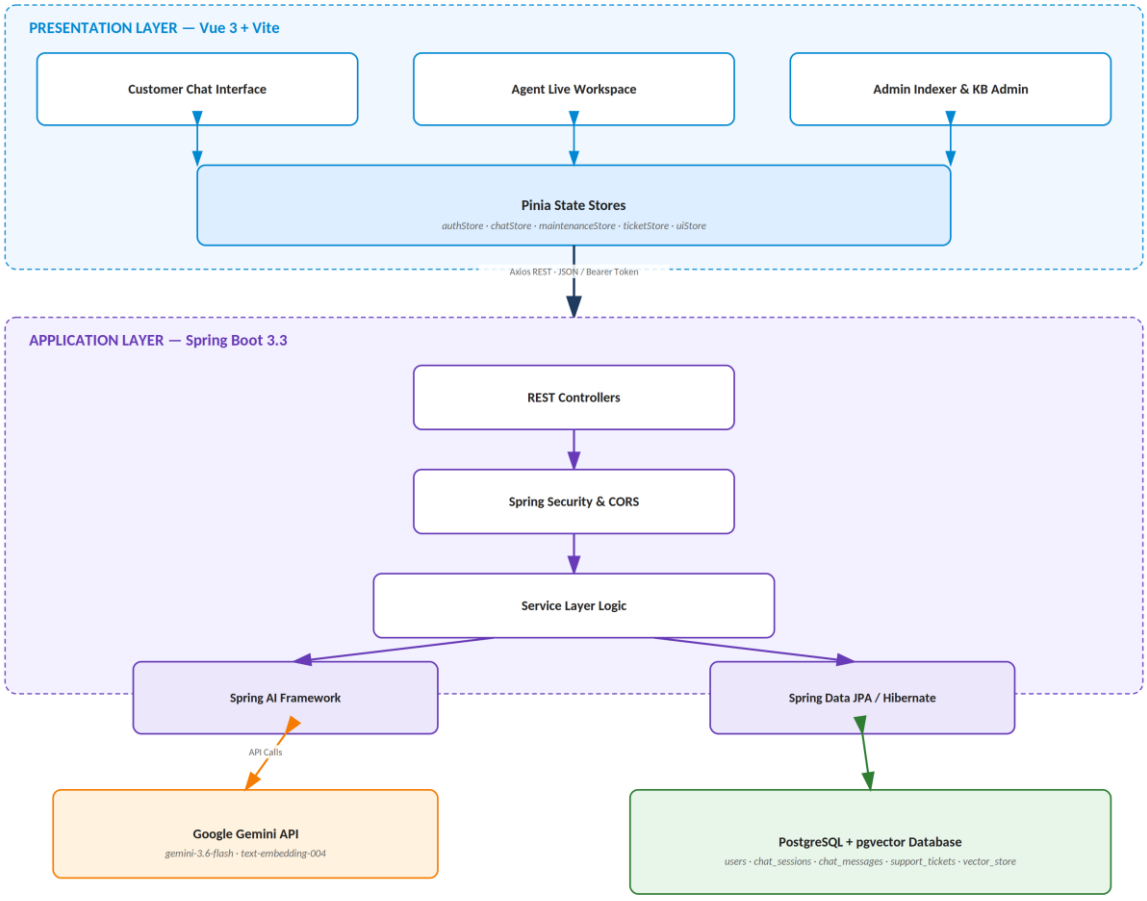

Figure 1 — High-Level System Architecture

2.1 Backend Architecture (Spring Boot 3.3+)

- **Framework & Runtime:** Powered by Spring Boot 3.3+, utilizing Java 17/21 features (Records, Pattern Matching, Sealed Classes) for clean domain models and efficient execution.
- **Data Access & Persistence:** Built on Spring Data JPA and Hibernate for ORM execution, with database migrations managing the following key relational entities:
  - users — role-based profile records (Admin, Support Agent, Customer)

- chat_sessions — distinct conversational sessions, metadata, and timestamps
- chat_messages — message history, roles (system, user, assistant), and execution logs
- support_tickets — handoff workflows when AI escalation is triggered
- system_tools & vector tables — tool availability tracking and domain indexing

Database Schema — Entity Model

Table Name	Primary Role	Key Attributes
users	Role-based profile management	id, email, password_hash, role (ADMIN, AGENT, CUSTOMER)
chat_sessions	Conversational session tracking	id, user_id, created_at, status
chat_messages	Message transaction history	id, session_id, sender_type, content, timestamp
support_tickets	Handoff escalation workflows	id, session_id, status (OPEN, IN_PROGRESS, RESOLVED), summary
system_tools	Domain tool tracking & indexer	id, name, status (AVAILABLE, BORROWED, IN_MAINTENANCE)
vector_store	Embeddings for RAG search	id, content_chunk, embedding_vector, metadata

- **AI & RAG Pipeline:** Leverages Spring AI connected directly to the Google Gemini API (gemini-3.6-flash / text-embedding-004), supporting document chunking, embeddings generation, and dynamic prompt contextualization.
- **Security & Resilience:** Configured with Spring Security for endpoint-level authorization, explicit CORS mappings (/api/v1/**, /api/tools/**), and centralized exception handlers intercepting HTTP 401, 403, 404, and 500 status responses.

2.2 Frontend Architecture (Vue 3 + Vite)

- **View Layer:** Vue 3 Composition API using script setup syntax, bundled with Vite for fast build performance. The UI is modularized into 15+ targeted components across three workspaces:
  - Customer Workspace — chat client interface with real-time message rendering, stream handling, and session management
  - Agent Workspace — real-time ticket queue processing, live chat intervention, and customer profile context
  - Admin Workspace — system indexer, knowledge base ingestion interface, user management, and operational analytics
- **State Management:** Pinia stores are split logically across six modules (agentStore, maintenanceStore, ticketStore, authStore, chatStore, uiStore) to manage local state, caching, and reactive UI triggers.
- **HTTP Service Gateway:** Modular Axios service wrappers (a publicClient for open endpoints and authenticated clients carrying Bearer tokens) decouple UI components from network logic.

3. Core Functional Capabilities

3.1 Retrieval-Augmented Generation (RAG) & Vector Indexing

The system moves beyond standardized conversational models by implementing context-driven vector search, following this pipeline:

Stage	Description
1. Ingestion	Documents and domain tool states are ingested into the pipeline.
2. Text Chunking	Source content is broken into indexable chunks.
3. Embedding	Chunks are converted into vector embeddings via Google's text-embedding-004 model.
4. Storage	Embeddings are written into PostgreSQL enhanced with the pgvector extension.
5. Retrieval	On query submission, cosine similarity search identifies top-matching knowledge chunks.
6. Generation	Matched context is injected into the system prompt before calling gemini-3.6-flash.

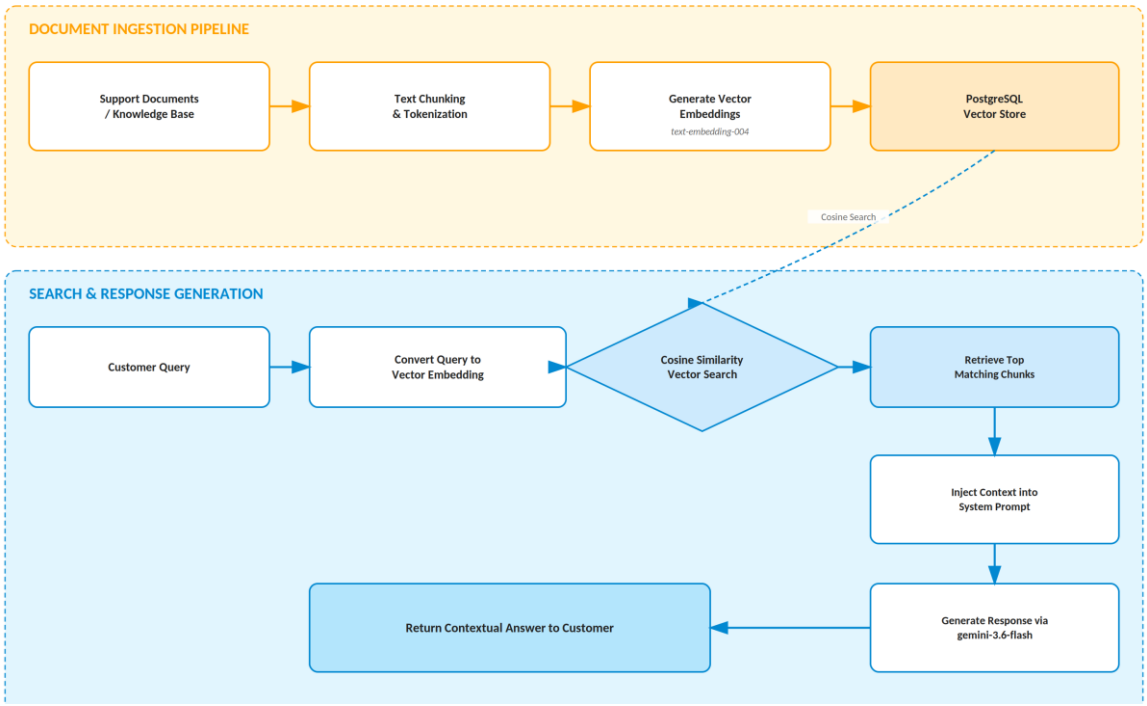

Figure 2 — RAG Vector Ingestion & Query Retrieval

3.2 AI-Assisted Handoff & Ticket Escalation

When the AI assistant encounters intent ambiguity, explicit customer requests, or low similarity confidence scoring, an escalation protocol is triggered automatically:

- An AI-generated summary of the preceding conversation is synthesized

- A structured entry is pushed to support_tickets for agent acquisition
- The ticket becomes visible in the Live Customer Workspace and Ticket Queue for seamless human handoff

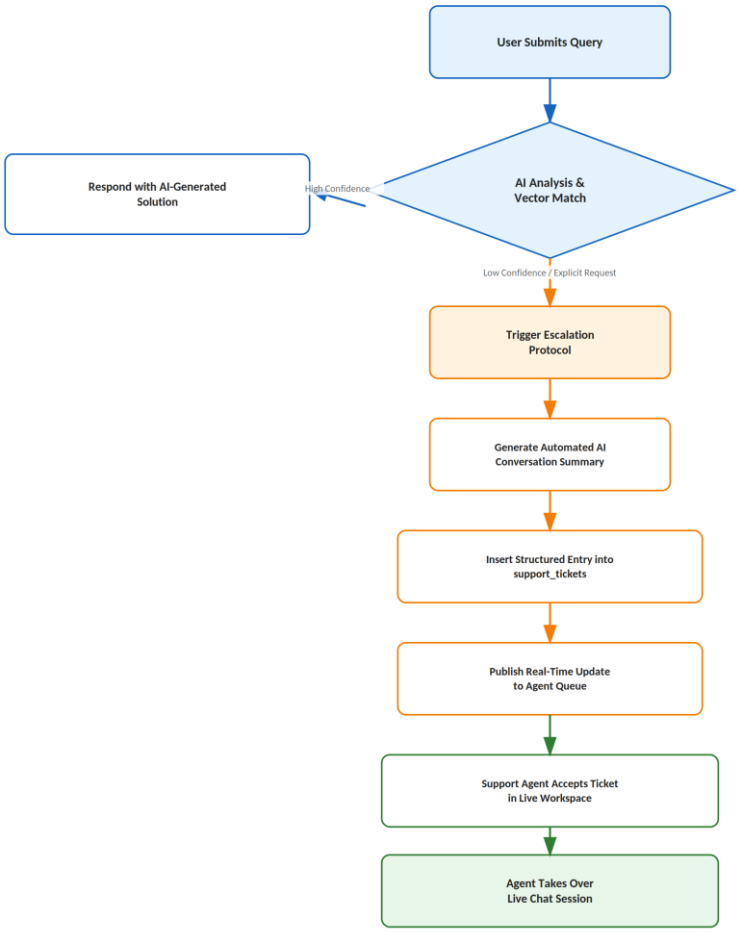

Figure 3 — AI-to-Human Handoff & Escalation Workflow

4. DevOps, Testing & Security Standards

Containerization

Fully containerized setup via docker-compose.yml, orchestrating PostgreSQL with pgvector enabled, pre-configured database ports, and persistent volumes.

CI/CD Integration

An integrated GitHub Actions workflow runs automated linting, test suites, and build validation on every push and pull request.

Testing Strategy

Layer	Tooling	Coverage
Backend	Spring Boot Test (@SpringBootTest, @WebMvcTest)	Repository interfaces, service contracts, controller mappings
Frontend	Vitest & Vue Test Utils	Component unit tests, store state mutation checks, utility validations

Integrations

Automated email transmission via Mailtrap/SMTP supports system notifications, ticket status updates, and user alerts.

5. Immediate & Upcoming Roadmap

Phase 1 — Completed

- Core full-stack architecture setup
- PostgreSQL + pgvector schema provisioning
- RAG pipeline integration with the Gemini API
- Admin, Agent, and Customer Vue 3 UI build
- Backend tool indexing

Phase 2 — In Progress / Next Steps

- Advanced analytics dashboard for tracking average AI deflection rates vs. human escalation rates
- WebSockets integration (Spring WebSocket / STOMP) for instant live agent-to-customer messaging without polling dependencies
- Granular RBAC permissions management inside the Knowledge Base Admin workspace

6. Installation & Deployment Quickstart

Prerequisites

- Java JDK 17 or 21
- Node.js 18+ and npm
- Docker Desktop & Docker Compose

Step 1 — Launch Database Infrastructure

Start the PostgreSQL container with pgvector pre-installed:

```
docker-compose up -d
```

Step 2 — Build & Start the Spring Boot Backend

Clean-compile and start the backend, providing your Gemini API key as an environment variable:

```
.\mvnw.cmd clean compile
$env:GEMINI_API_KEY="YOUR_GEMINI_API_KEY_HERE"
.\mvnw.cmd spring-boot:run
```

Step 3 — Launch the Vue 3 Frontend Development Server

Navigate to the frontend directory, install dependencies, and run the dev server:

```
cd frontend
npm install
npm run dev
```

Access Points

Service	URL
Frontend (Vite Dev Server)	http://localhost:5173
Backend API Base	http://localhost:8080/api/v1

Prepared by Arnold Niyonkuru — CODAFRIQA